

Informe técnico integral de la arquitectura híbrida

Ecommify

Autores:

Oscar Clavijo, Daniel Porras, Camilo Porras

Universidad de la Sabana

Diseño y Optimización de Base de datos

Docente: Miguel Alfonso Varela

23 de junio de 2026

Resumen

Este informe consolida el diseño, implementación, optimización y evaluación crítica de la arquitectura híbrida de bases de datos implementada para Ecommify, una plataforma de comercio electrónico con necesidades transaccionales, documentales y analíticas. La solución combina PostgreSQL/Supabase para datos críticos que requieren consistencia fuerte, integridad referencial y soporte ACID, con MongoDB Atlas para catálogo flexible, reseñas, comportamiento de usuario, recomendaciones y logs de búsqueda. El proyecto utilizó el dataset Brazilian E-Commerce Public Dataset by Olist como fuente semilla, transformándolo hacia un modelo de datos propio de Ecommify.

Los resultados muestran que PostgreSQL fue más adecuado para procesos transaccionales y reportes con consistencia fuerte, mientras que MongoDB ofreció mayor flexibilidad para datos semiestructurados y servicios derivados. La evaluación de rendimiento evidenció mejoras relevantes con índices especializados y particionamiento, destacando la búsqueda textual con `pg_trgm`, que mejoró 85.70 %, y la consulta geoespacial con PostGIS, que mejoró 67.21 %. La prueba de concurrencia mostró mejor throughput de PostgreSQL en escenarios controlados, aunque también permitió identificar un punto de quiebre operativo del entorno Supabase con 25 usuarios concurrentes.

Palabras clave: arquitectura híbrida, PostgreSQL, MongoDB, Ecommify, CAP, rendimiento, escalabilidad.

Tabla de contenido

Resumen	2
Tabla de contenido	¡Error! Marcador no definido.
1. Executive Summary.....	¡Error! Marcador no definido.
2. Introducción y contexto	7
2.1 Problema de negocio	7
2.2 Objetivos y alcance	7
2.3 Metodología aplicada	8
3. Arquitectura y diseño	9
3.1 Matriz de decisión de persistencia	¡Error! Marcador no definido.
3.2 Teorema CAP por módulo	¡Error! Marcador no definido.
3.3 Modelos sintéticos	¡Error! Marcador no definido.
4. Implementación técnica	19
4.1 Implementación PostgreSQL	19
4.2 Implementación MongoDB.....	19
4.3 Sincronización PostgreSQL hacia MongoDB.....	20
5. Evaluación de rendimiento y escalabilidad	20
5.1 Resultados PostgreSQL antes y después de optimización.....	20
5.2 Pruebas de carga concurrente	22
5.3 Pruebas de escalabilidad con datasets crecientes	23

5.4 Punto de quiebre y limitaciones	24
6. Análisis arquitectónico crítico	24
6.1 Evaluación de selección tecnológica	24
6.2 Consistencia eventual y mitigación	25
6.3 Escenarios de falla.....	25
6.4 Reflexión y lecciones aprendidas.....	25
7. Recomendaciones estratégicas.....	26
7.1 Plan de escalamiento 10x	26
7.2 Migración de free tier a producción	26
7.3 Tecnologías complementarias	27
7.4 Estrategia CI/CD para cambios de esquema	27
8. Conclusiones	27
9. Referencias	28
10. Anexos.....	29

1. Resumen ejecutivo de la solución

Ecommify fue planteada como una plataforma de comercio electrónico con requerimientos heterogéneos de persistencia: operaciones transaccionales críticas, catálogo flexible, analítica de comportamiento, recomendaciones y trazabilidad. Para responder a estas necesidades se implementó una arquitectura híbrida basada en PostgreSQL/Supabase y MongoDB Atlas. PostgreSQL se utilizó para usuarios, direcciones, órdenes, pagos, inventario, envíos y auditoría transaccional. MongoDB se utilizó para catálogo documental, reseñas, comportamiento de usuario, recomendaciones y logs de búsqueda.

La arquitectura implementada logró transformar el dataset Olist hacia un modelo propio de Ecommify. En PostgreSQL se cargaron 99.441 usuarios, 99.441 órdenes, 112.650 detalles de órdenes, 32.951 productos y 198.881 eventos transaccionales. En MongoDB se consolidaron 32.951 documentos de catálogo, 102.172 reseñas, 112.650 eventos de comportamiento, 1.000 recomendaciones y 5.000 logs de búsqueda.

Las métricas de optimización evidenciaron mejoras significativas en consultas críticas. La búsqueda textual con pg_trgm mejoró 85.70 %, la consulta geoespacial con PostGIS mejoró 67.21 %, el historial de órdenes por usuario mejoró 63.61 %, los pagos aprobados por fecha mejoraron 62.27 % y la auditoría por eventos mejoró 58.44 %. Estos resultados confirman que el uso de índices especializados, particionamiento declarativo y modelado orientado al caso de uso genera beneficios medibles.

La recomendación estratégica principal es mantener la arquitectura híbrida. Para una migración a producción se recomienda incorporar PgBouncer, réplicas de lectura, Redis, OpenSearch, monitoreo centralizado y un flujo de sincronización basado en

eventos mediante Kafka, Debezium o Change Data Capture. La sincronización batch implementada es válida como aproximación académica, pero en producción debe evolucionar hacia procesamiento asíncrono observable, tolerante a fallos e idempotente.

Tabla 1. Conteos finales del modelo relacional PostgreSQL.

Tabla PostgreSQL	Registros
users	99441
addresses	99441
categories	74
products	32951
inventories	32951
carts	500
orders	99441
order_details	112650
payments	99440
shipments	99441
reviews	101628
transaction_history	198881
sync_audit	1

Tabla 2. Conteos finales del modelo documental MongoDB.

Colección MongoDB	Documentos
product_catalog	32951
product_reviews	102172
user_behavior	112650
recommendations	1000
search_logs	5000
sellers	3095

2. Introducción y contexto

2.2 Problema de negocio

Ecommify requiere soportar procesos típicos de comercio electrónico: registro de usuarios, administración de productos, inventario, carritos, órdenes, pagos, envíos, reseñas, búsquedas, recomendaciones y análisis del comportamiento de los usuarios. El problema central consiste en que no todos estos datos tienen las mismas necesidades de consistencia, flexibilidad, disponibilidad y escalabilidad.

Los módulos de pagos, órdenes e inventario no admiten inconsistencias, ya que pueden generar impactos financieros, sobreventa o pérdida de trazabilidad. Por el contrario, módulos como recomendaciones, logs de búsqueda o comportamiento de usuario pueden aceptar consistencia eventual, siempre que la experiencia de navegación y lectura se mantenga disponible.

2.3 Objetivos y alcance

- Diseñar una arquitectura híbrida PostgreSQL + MongoDB para Ecommify.
- Implementar un modelo relacional optimizado para datos transaccionales críticos.
- Implementar un modelo documental flexible para catálogo, reseñas, comportamiento, recomendaciones y búsquedas.
- Ejecutar pruebas de rendimiento, concurrencia y escalabilidad.
- Comparar PostgreSQL y MongoDB con evidencia empírica.
- Analizar decisiones arquitectónicas mediante el Teorema CAP.
- Proponer recomendaciones estratégicas para producción y crecimiento 10x.

2.4 Metodología aplicada

La metodología integró diseño conceptual, implementación técnica, carga de datos, optimización, pruebas empíricas y análisis crítico. El dataset Olist se utilizó como fuente semilla, pero no como modelo final. Los datos fueron transformados hacia entidades relacionales y colecciones documentales alineadas con el diseño de Ecommify. Posteriormente se ejecutaron mediciones mediante EXPLAIN ANALYZE, scripts de concurrencia, pruebas de escalabilidad y análisis CAP por módulo.

3. Arquitectura y diseño

Este documento define el diseño de una arquitectura híbrida de persistencia para Ecommify, combinando PostgreSQL y MongoDB. La propuesta se optimizó luego de obtener las evidencias cuantitativas obtenidas en las pruebas de optimización realizadas sobre ambas tecnologías: índices, particionamiento, pipelines de agregación y planes de ejecución (EXPLAIN ANALYZE).

El criterio de diseño central es la consistencia requerida por cada dominio de datos: los datos que afectan dinero, inventario e identidad permanecen en PostgreSQL bajo garantías ACID; los datos de catálogo, comportamiento y experiencia de usuario se sirven desde MongoDB, donde la consistencia eventual es aceptable y el rendimiento de lectura/escritura masiva es superior.

3.2 Diagrama de arquitectura híbrida completa

La arquitectura se organiza en cuatro capas: clientes, una capa de enrutamiento (API Gateway / BFF) que decide el destino de cada operación según su naturaleza transaccional o de lectura, los dos motores de persistencia con sus topologías de alta disponibilidad, y un canal de sincronización basado en Change Data Capture (CDC) que mantiene MongoDB actualizado a partir de PostgreSQL como fuente de verdad.

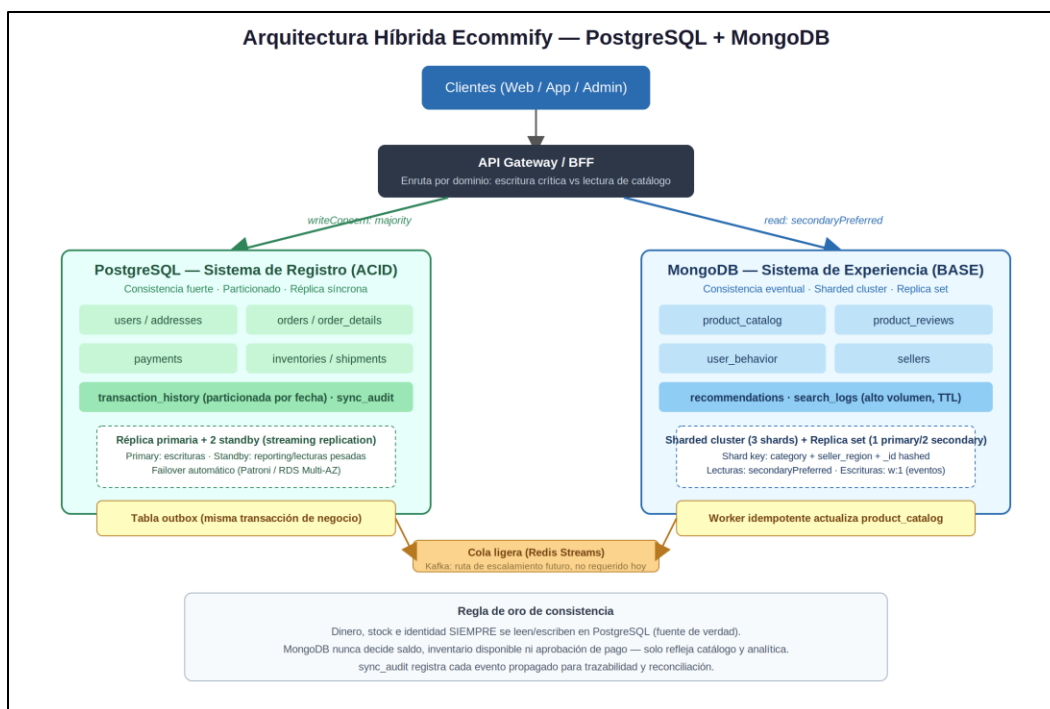


Figura 1. Arquitectura híbrida Ecommify: enrutamiento por dominio, topologías de alta disponibilidad y sincronización vía outbox + worker con cola ligera.

3.3 Componentes clave

- **API Gateway / BFF:** enruta comandos críticos (pedidos, pagos, stock) hacia PostgreSQL con writeConcern de mayoría, y lecturas de catálogo/recomendación hacia MongoDB con secondaryPreferred.
- **Patrón Outbox transaccional:** cuando la app modifica products, inventories u orders, en la misma transacción de base de datos inserta una fila en la tabla outbox. Esto garantiza que el evento de sincronización nunca se pierde ni queda "a medias", porque o se confirman juntos el cambio de negocio y el evento, o no se confirma ninguno.
- **Worker de sincronización + cola ligera (Redis Streams):** un proceso liviano lee la tabla outbox, publica el evento en una cola en memoria y lo entrega a un consumidor que actualiza Mongo. No se usa Kafka: para el volumen real de Ecommify, Kafka agrega una complejidad operativa y un costo mensual que no se justifican (ver sección 3.1, Justificación de costos).
- **Worker idempotente del lado MongoDB:** aplica los eventos sobre product_catalog y colecciones derivadas, tolerando reintentos mediante una clave de versión, sin necesidad de un cluster de mensajería pesado.

- `sync_audit` (PostgreSQL): bitácora de cada evento propagado, su estado de sincronización y reintentos, habilitando reconciliación y auditoría.
- Réplicas: PostgreSQL con streaming replication (1 primary + 2 standby) para alta disponibilidad transaccional; MongoDB con cluster fragmentado (sharded) de 3 shards y replica set por shard para escalar lectura/escritura de catálogo y eventos.

3.4 Estrategia de sincronización entre PostgreSQL y MongoDB

PostgreSQL y MongoDB no son dos bases aisladas: el catálogo, las reseñas y los datos de comportamiento que vive en MongoDB se alimentan de cambios que ocurren primero en PostgreSQL (la fuente de verdad de productos, inventario y órdenes). Por eso se necesita un mecanismo que propague esos cambios de forma confiable, sin que un fallo en Mongo bloquee una venta, y sin perder eventos en el camino.

La estrategia elegida combina dos ideas simples: Outbox transaccional (garantiza que el evento nunca se pierde) y un worker con cola ligera (garantiza que el evento llegue a Mongo sin necesitar infraestructura pesada como Kafka).

3.5 Justificación de costos: ¿por qué sin Kafka?

Se evaluaron dos caminos para mover los eventos desde PostgreSQL hacia MongoDB: usar Kafka (con Debezium leyendo los cambios) o usar un worker simple con una cola ligera como Redis Streams. La diferencia no es solo técnica, también es de costo operativo, y para el volumen real de Ecommify la opción más simple es la más razonable.

Aspecto	Con Kafka + Debezium	Sin Kafka (Outbox + worker/Redis Streams)
Infraestructura mínima	Cluster de al menos 3 nodos (broker) más el conector de Debezium	Un worker (proceso pequeño) y, opcionalmente, un Redis económico
Costo mensual estimado	Entre 150 y 1.000+ USD/mes según el proveedor (self-hosted o administrado)	0 a 15 USD/mes si se usa Redis administrado; puede ir incluso sin costo adicional
Conocimiento especializado requerido	Alto: particiones, consumer groups, offsets, rebalanceo	Bajo: es código de aplicación y una cola estándar
Throughput que soporta	Decenas de miles a millones de eventos por segundo	Miles de eventos por segundo, suficiente para el volumen de Ecommify
Cuando se justifica	Cuando varios sistemas distintos (analítica, notificaciones, fraude) necesitan leer el mismo flujo de eventos de forma independiente	Cuando el único consumidor real del evento es la sincronización del catálogo, como es el caso actual

Figura 2. La recomendación es iniciar con Outbox + worker, y dejar Kafka documentado como ruta de escalamiento si el negocio crece a un punto donde varios sistemas necesitan consumir el mismo flujo de eventos.

3.6 Cómo funciona el flujo paso a paso

1. La aplicación actualiza products, inventories u orders en PostgreSQL.
2. En la misma transacción, inserta una fila en la tabla outbox con el evento (qué cambió y su contenido).
3. Un worker lee la tabla outbox, publica el evento en una cola ligera (Redis Streams) y lo marca como enviado.
4. Otro proceso (o el mismo worker) toma el evento de la cola y actualiza la colección correspondiente en MongoDB, de forma idempotente para tolerar reintentos.
5. Cada paso queda registrado en sync_audit, lo que permite detectar y reconciliar cualquier evento que no se haya aplicado correctamente.

3.7 ¿Cómo se soportaría alta transaccionalidad como un Black

Friday?

Siempre que se tengan en cuenta algunos puntos que no son automáticos y que deben quedar explícitos en el diseño. La idea central es que un pico de tráfico afecta la

velocidad con la que el catálogo se actualiza, pero nunca debe afectar la seguridad de una venta, porque esa responsabilidad sigue siempre en PostgreSQL.

Riesgo en el pico de tráfico	Cómo se mitiga
La tabla outbox crece muy rápido	Se limpia tan pronto el worker confirma el envío; no es una tabla de auditoría permanente, solo un buffer de paso
El worker se queda atrás procesando eventos	La cola (Redis Streams) absorbe el volumen acumulado; el efecto es que el catálogo tarda un poco más en reflejar el cambio, no que se pierda información
Un solo worker no alcanza a procesar todo el volumen	Se pueden correr varios workers en paralelo, repartiendo el trabajo por producto, sin necesitar un cluster de mensajería más grande
MongoDB recibe una ráfaga de escrituras de catálogo	El diseño de sharding (categoría + región + id) ya reparte esa carga entre los tres shards en vez de concentrarla en uno solo
Muchos usuarios navegando el catálogo al mismo tiempo	Las lecturas se atienden desde nodos secundarios (secondaryPreferred), sin competir con las escrituras
El worker o la cola fallan por completo durante el pico	El catálogo en Mongo simplemente se queda con el último estado conocido (degradación controlada); sigue funcionando para mostrar productos, solo que no refleja los cambios más recientes

Figura 3. Riesgos en el pico de tráfico y sus mitigaciones

3.8 Matriz de decisiones: PostgreSQL vs MongoDB

La matriz siguiente resume, para cada dominio de datos de Ecommify, la tecnología asignada y la justificación basada en las pruebas de rendimiento documentadas en el repositorio del proyecto.

Dominio de datos	Tecnología	Justificación basada en evidencia
Usuarios y autenticación	PostgreSQL	Requiere integridad referencial y unicidad (email). No es un dato de alto volumen variable.
Direcciones (geolocalización)	PostgreSQL	Q6 geoespacial con PostGIS mejoró 67.21% con índice GiST; tipo geography requiere motor relacional con soporte espacial maduro.
Órdenes y detalle de orden	PostgreSQL	Q1 (historial por usuario) mejoró 63.61% con B-tree + partición; requiere ACID por ser el registro legal de la venta.
Pagos	PostgreSQL	Q7 mejoró 62.27% con partición + BRIN; dato financiero sensible, exige writeConcern fuerte y trazabilidad.
Inventario	PostgreSQL	Decisiones de stock disponible no toleran inconsistencia eventual (riesgo de sobreventa).
Historial transaccional / auditoría	PostgreSQL	Q8 mejoró 58.44% con partición + índices B-tree/BRIN; auditoría exige consistencia fuerte.
Catálogo de productos	MongoDB	Índice ESR sobre category+region+status+rating+price logró hasta 96.3% de mejora; esquema flexible para atributos variables por categoría.
Reseñas de productos	MongoDB	Índice por product_id logró 98.97% de mejora; pipeline optimizado de agregación redujo 62.27% el tiempo (959→362 ms).
Comportamiento de usuario (eventos)	MongoDB	Alto volumen de escritura, tolera consistencia eventual; aunque el índice event_type+period no fue productivo (-7.95%), el modelo documental sigue siendo el adecuado para el volumen.
Vendedores (sellers)	MongoDB	Índice region+city logró 97.78% de reducción de documentos examinados; dato de catálogo, no transaccional.
Recomendaciones / búsqueda	MongoDB	Resultados recalculables, no requieren durabilidad estricta; consistencia eventual aceptada explícitamente en el diseño de read preference.

Figura 4. Casos límite documentados: las consultas Q3 y Q4 en PostgreSQL (filtros sobre JSONB y ARRAY) no mejoraron con índices GIN debido a baja selectividad, igual que el índice de user_behavior en MongoDB. Esto confirma que la asignación de tecnología no exim

3.9 Análisis del teorema CAP aplicado por módulo

El teorema CAP establece que, ante una partición de red (P), un sistema distribuido debe priorizar Consistencia (C) o Disponibilidad (A). La siguiente tabla aplica

este trade-off a cada módulo de Ecommify, indicando la prioridad elegida y su mecanismo concreto.

Módulo	Tecnología	Prioridad CAP	Mecanismo / consecuencia
Pagos	PostgreSQL	CP	writeConcern/transacción ACID; ante partición, se rechaza la operación antes que confirmar un pago inconsistente.
Órdenes e inventario	PostgreSQL	CP	Lecturas desde Primary para evitar vender stock inexistente; failover automático prioriza consistencia sobre continuidad inmediata.
Usuarios / autenticación	PostgreSQL	CP	Unicidad de email y password_hash no admiten duplicidad bajo partición.
Auditoría / transaction_history	PostgreSQL	CP	Trazabilidad legal exige orden y completitud, no disponibilidad a toda costa.
Catálogo de productos	MongoDB	AP	secondaryPreferred: ante partición, se sirve catálogo posiblemente desactualizado en vez de error; el precio autoritativo sigue en PostgreSQL.
Reseñas	MongoDB	AP	Lectura tolera staleness; escritura usa majority solo porque es visible a otros usuarios, pero no bloquea el flujo de compra.
Comportamiento de usuario / eventos	MongoDB	AP	writeConcern w:1: se prioriza disponibilidad de ingesta; pérdida ocasional de un evento es aceptable y reprocesable.
Recomendaciones / search_logs	MongoDB	AP	Resultados recalculables; ante partición se sirve la última versión disponible (secondary).
Sincronización Postgres → Mongo	Outbox + worker	AP con consistencia eventual acotada	sync_audit permite detectar y reconciliar divergencias; el catálogo nunca es la fuente de verdad del precio o stock.

Figura 5. Teorema CAP

3.10 Mitigaciones de consistencia eventual

- Operaciones críticas (pago, confirmación de orden, actualización de precio) siempre leen y escriben en el Primary de PostgreSQL con writeConcern majority.
- Eventos hacia MongoDB son idempotentes (clave de versión/timestamp) para tolerar reintentos del consumer ante caídas.

3.11.1 Modelo Entidad-Relación — PostgreSQL

El modelo relacional normaliza las entidades transaccionales. Las tablas `orders`, `payments` y `transaction_history` están particionadas declarativamente por fecha.

`sync_audit` es la tabla de control de la sincronización hacia MongoDB.

Figura 6. Modelo entidad-relacion PostgreSQL

3.12 Modelo de documentos — MongoDB

Las colecciones documentales priorizan datos autocontenidos y embebidos para minimizar joins en lectura. El `_id` de `product_catalog` se ancla al uuid canónico generado en PostgreSQL, evitando depender del identificador de un dataset de origen.

product_catalog _id (mixed) attributes () description_le. (mixed) height_cm (mixed) length_cm (mixed) name_length (mixed) photos_qty (mixed) seller_city string seller_id string seller_state string source_category (mixed) source_product_ string weight_g (mixed) width_cm (mixed) category string created_at date currency string description string images () inventory () available_stock int reserved_stock int stock int metrics () conversion_rate double total_revenue double total_units_so. int views_count int name string price double product_id string ratings () average double count int recent_reviews () seller_city string seller_id string seller_region string source (mixed) dataset string original_produ. string status string synced_at date tags () updated_at date

 product_reviews _id (mixed) answer_timestamp date comment (mixed) created_at date order_id string product_id string product_snap. () category string name string rating int review_id string source (mixed) dataset string synced_at date title string user_id string verified_purcha. bool | recommendations _id objectId algorithm () name string strategy string version string generated_at date recommenda. () category string name string product_id string reason string score double source string user_id string | search_logs _id objectId clicked_produ. null filters () category (mixed) max_price (mixed) min_rating (mixed) query string results_count int searched_at date source string user_id string | sellers _id string active bool city string country string created_at date region string seller_id string source () dataset string updated_at date | sync_audit _id objectId collections () executed_at date limit_rows_pe. int mode string source string status string sync_name string target string | user_behavior _id objectId created_at date event_count int events () entity_id string entity_name string event_date date event_type string metadata () order_id int source_ord. string status string total double user_id int productId string timestamp date type string period string source (mixed) dataset string order_id string updated_at date user_id string userId string |

Figura 7. Modelo de documentos sintético de MongoDB, incluyendo la lección de selectividad observada en `user_behavior`.

3.13 Correlación entre modelos

Entidad PostgreSQL	Colección MongoDB	Clave de correlación
products.uuid	product_catalog._id	uuid canónico (sustituye al product_id de Olist)
orders.id / order_details	user_behavior.events[].productId	order_id referenciado en source.order_id
reviews (Postgres, post-compra verificada)	product_reviews	product_id + order_id
sync_audit	(meta) todas las colecciones sincronizadas	entity_name + entity_id + synced_at

Figura 8. correlación entre modelos

3.14 Conclusión

La arquitectura híbrida propuesta no introduce tecnología nueva sin respaldo: cada asignación de dominio a PostgreSQL o MongoDB está sustentada en una mejora porcentual medida (B-tree, GIN, GiST, BRIN, pg_trgm en Postgres; índices ESR y pipelines optimizados en Mongo), y cada caso donde un índice no fue productivo se documenta como restricción de diseño en lugar de ignorarse.

El punto que la arquitectura añade sobre la evidencia ya recolectada es el mecanismo de sincronización (outbox transaccional + worker con cola ligera + sync_audit), indispensable para que ambas bases dejen de ser silos cargados independientemente desde el mismo CSV y pasen a operar como un sistema único con una fuente de verdad clara para cada tipo de dato. Esta combinación se eligió deliberadamente por encima de una solución basada en Kafka: para el volumen real de Ecommify resuelve el mismo problema con una fracción del costo y la complejidad operativa, dejando Kafka como ruta de escalamiento si el negocio llega a un punto donde varios sistemas distintos necesiten consumir el mismo flujo de eventos.

4. Implementación técnica

4.2 Implementación PostgreSQL

PostgreSQL se implementó como repositorio transaccional principal. El modelo relacional incluye usuarios, direcciones, categorías, productos, inventarios, carritos, órdenes, detalles de orden, pagos, envíos, reseñas, historial transaccional y auditoría de sincronización. Se aplicaron restricciones de integridad, llaves foráneas, validaciones CHECK y estrategias de particionamiento para tablas de alto crecimiento.

Tabla 3. Técnicas implementadas en PostgreSQL.

Técnica	Aplicación	Justificación
JSONB	products.attributes, orders.metadata, transaction_history.payload	Atributos y metadatos flexibles.
TEXT[]	products.tags	Etiquetas de búsqueda y clasificación.
Composite type	addresses.address_data	Estructura compuesta de dirección.
PostGIS geography	addresses.location	Consultas geoespaciales.
pg_trgm	products.name y description	Búsqueda textual eficiente.
BRIN	orders, payments, transaction_history	Rangos de fecha en tablas grandes.
GIN	JSONB y arrays	Búsquedas en estructuras flexibles.
GIST	geography	Índices espaciales.
Particionamiento	orders, payments, transaction_history	Particiones mensuales por fecha.

4.3 Implementación MongoDB

MongoDB Atlas se implementó como repositorio documental y analítico. Las colecciones principales fueron product_catalog, product_reviews, user_behavior, recommendations y search_logs. También se conservó la colección sellers como auxiliar. El diseño documental permitió representar atributos variables, snapshots, eventos agrupados, búsquedas simuladas y recomendaciones derivadas.

Tabla 4. Patrones de modelado documental implementados en MongoDB.

Patrón	Colección	Uso
Attribute Pattern	product_catalog	Atributos variables por categoría de producto.
Extended Reference	product_reviews	Snapshot del producto dentro de la reseña.
Bucket Pattern	user_behavior	Agrupación de eventos por usuario y periodo.
Embedding	product_catalog	Ratings y reseñas resumidas.
Referencing	reviews, recommendations	Relaciones lógicas entre usuario, producto y evento.

4.4 Sincronización PostgreSQL hacia MongoDB

Se implementó una sincronización batch idempotente desde PostgreSQL hacia MongoDB. El proceso representa el flujo entre el modelo transaccional y el modelo documental. Para la evidencia académica se ejecutó en modo limitado de 1.000 registros por entidad, evitando timeouts del entorno administrado. La idempotencia se logró mediante operaciones UpdateOne con upsert=True, permitiendo repetir el proceso sin duplicar documentos.

Enfoque de decisiones: falta

5. Evaluación de rendimiento y escalabilidad

5.2 Resultados PostgreSQL antes y después de optimización

Tabla 5. Resultados de optimización PostgreSQL con EXPLAIN ANALYZE.

Consulta	Antes (ms)	Después (ms)	Mejora
Q1 Historial de órdenes por usuario	37.830	13.765	63.61%
Q2 Ventas por categoría y mes	2431.127	1405.278	42.20%
Q3 Productos por atributos JSONB	3.250	4.544	-39.82%
Q4 Productos por tags ARRAY	0.138	0.151	-9.42%

Q5 Búsqueda textual con pg_trgm	15.547	2.223	85.70%
Q6 Consulta geoespacial con PostGIS	700.755	229.750	67.21%
Q7 Pagos aprobados por fecha	733.974	276.963	62.27%
Q8 Historial transaccional por evento	1298.803	539.737	58.44%

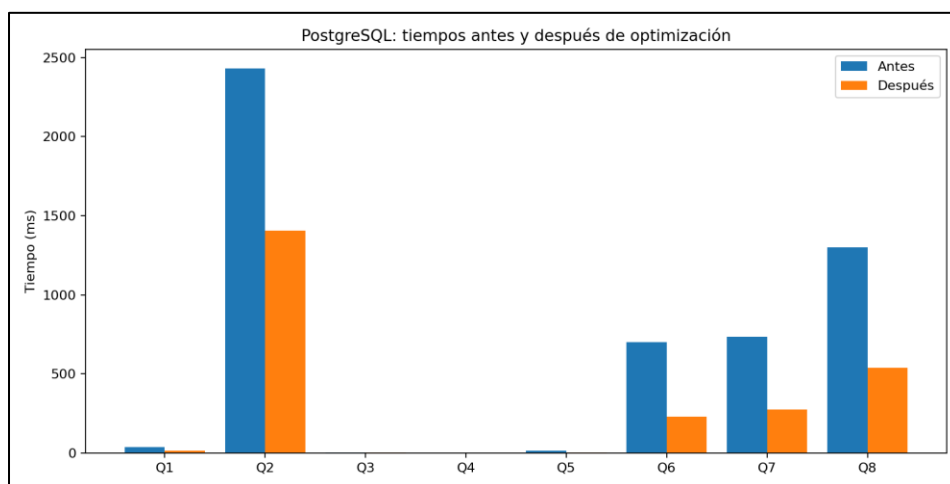


Figura 9. Comparación de tiempos PostgreSQL antes y después de la optimización

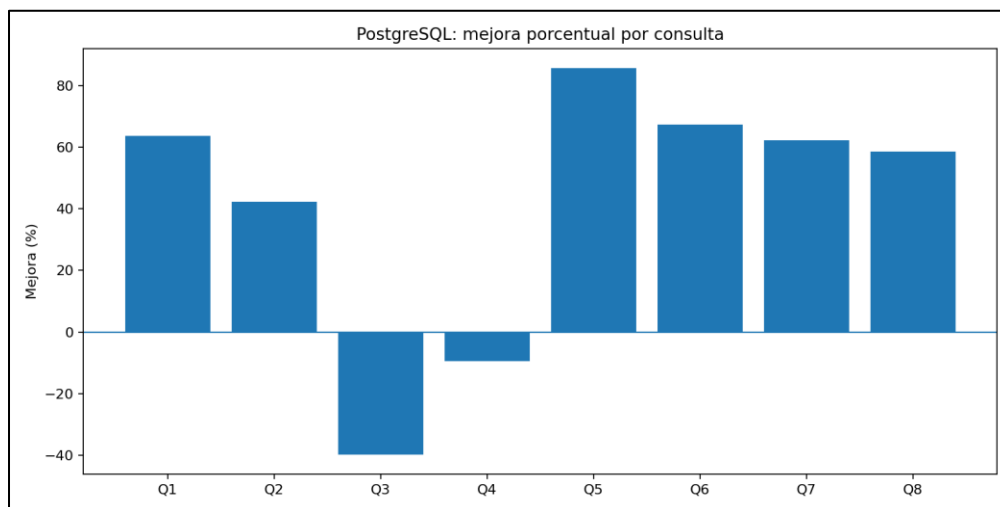


Figura 10. Porcentaje de mejora por consulta PostgreSQL.

Las mayores mejoras se obtuvieron en búsqueda textual con pg_trgm, consulta geoespacial con PostGIS, historial de órdenes por usuario, pagos por fecha y eventos de auditoría. Las consultas Q3 y Q4 no mejoraron debido a baja selectividad o ausencia de coincidencias, lo cual evidencia que un índice solo es efectivo cuando el patrón de consulta lo justifica.

5.3 Pruebas de carga concurrente

Tabla 6. Resultados de pruebas de concurrencia.

Motor	Usuarios	Requests	Avg (ms)	P95 (ms)	Max (ms)	Req/s
PostgreSQL	1	10	531.14	590.15	573.46	1.88
MongoDB	1	10	728.96	866.35	825.29	1.37
PostgreSQL	5	50	567.21	658.80	672.28	8.59
MongoDB	5	50	730.70	782.26	790.94	6.82
PostgreSQL	10	100	756.17	966.00	981.94	12.87
MongoDB	10	100	804.16	922.33	937.31	12.35

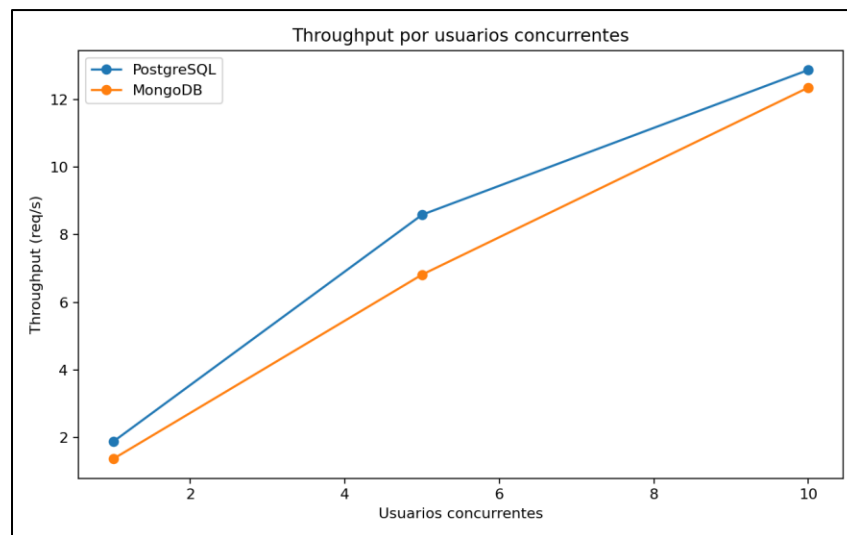


Figura 11. Throughput por usuarios concurrentes.

PostgreSQL presentó menor latencia promedio y mayor throughput en los escenarios controlados de 1, 5 y 10 usuarios concurrentes. La diferencia se redujo en

el escenario de 10 usuarios, donde MongoDB mostró un throughput cercano al de PostgreSQL.

5.4 Pruebas de escalabilidad con datasets crecientes

Tabla 7. Resultados de escalabilidad con datasets crecientes.

Dataset	PostgreSQL (ms)	MongoDB (ms)
1000	1775.04	749.62
5000	533.00	664.17
10000	527.38	690.64
25000	535.36	716.42
50000	531.35	771.77

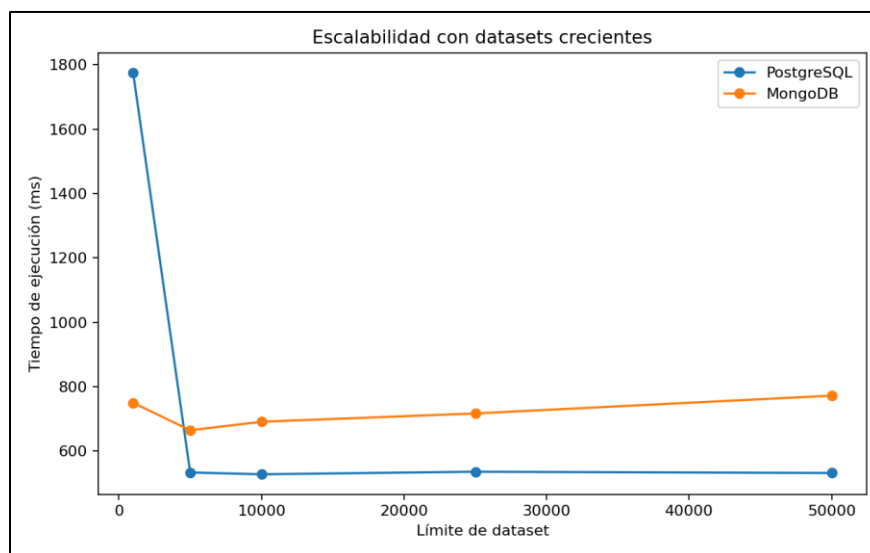


Figura 12. Escalabilidad con datasets crecientes.

MongoDB fue más rápido en el primer escenario de 1.000 registros, mientras que PostgreSQL mostró mayor estabilidad desde 5.000 hasta 50.000 registros. El resultado inicial de PostgreSQL puede estar influenciado por calentamiento de conexión, planificación o caché. A partir de 5.000 registros, el tiempo de PostgreSQL se mantuvo cercano a 530 ms.

5.5 Punto de quiebre y limitaciones

Durante la prueba con 25 usuarios concurrentes en PostgreSQL/Supabase se presentó un cierre inesperado de la conexión SSL con error EDBHANDLEREXITED. Este resultado se documentó como punto de quiebre operativo del entorno administrado. La causa probable está asociada a límites de conexiones concurrentes, recursos disponibles, timeouts o configuración del proveedor. Este hallazgo demuestra que la escalabilidad depende tanto del motor de base de datos como de la configuración operacional.

Tabla 8. Cuellos de botella identificados.

Cuello de botella	Evidencia	Recomendación
Límite de conexiones en Supabase	Error con 25 usuarios concurrentes	PgBouncer o plan superior.
Dashboards SQL pesados	Q2 optimizada aún tarda 1405 ms	Vistas materializadas o agregados.
Baja selectividad GIN	Q3 y Q4 con mejora negativa	Rediseñar filtros y atributos.
Latencia agregada en MongoDB	700 a 800 ms en algunas pruebas	Índices compuestos y sharding.
Sync batch pesado	Timeout inicial en bulk_write	Lotes, CDC o eventos.

6. Análisis arquitectónico crítico

6.2 Evaluación de selección tecnológica

La selección tecnológica fue adecuada. PostgreSQL debe mantenerse para pagos, órdenes, inventario y usuarios, porque estos módulos requieren consistencia fuerte, auditoría e integridad referencial. MongoDB debe mantenerse para catálogo enriquecido, comportamiento, recomendaciones, reseñas y logs, porque estos dominios requieren flexibilidad, disponibilidad y tolerancia a crecimiento documental.

Los productos y las reseñas son módulos naturalmente híbridos. PostgreSQL conserva el producto normalizado e inventariado, mientras MongoDB mantiene el

catálogo enriquecido para lectura y experiencia de usuario. En reseñas, PostgreSQL permite validar reglas de negocio asociadas a compras, mientras MongoDB facilita la visualización y analítica documental.

6.3 Consistencia eventual y mitigación

Tabla 9. Ventanas de consistencia eventual y mitigación.

Flujo	Ventana	Riesgo	Mitigación
PostgreSQL -> product_catalog	Segundos o minutos	Catálogo desactualizado	Sync idempotente y reintentos.
Orders -> recommendations	Minutos	Recomendaciones antiguas	Batch frecuente o eventos.
Search logs	Eventual	Pérdida o retraso analítico	Cola persistente y DLQ.
Reviews -> MongoDB	Segundos o minutos	Reseña no visible	Reintentos y auditoría.

6.4 Escenarios de falla

Tabla 10. Escenarios de falla y estrategias de mitigación.

Escenario	Impacto	Mitigación
MongoDB no disponible	No hay recomendaciones, logs o catálogo enriquecido	Continuar transacciones en PostgreSQL.
PostgreSQL no disponible	No se deben permitir compras, pagos ni reservas	Bloquear checkout hasta recuperar consistencia.
Network partition	MongoDB puede quedar desactualizado	Reintentos, batch idempotente o eventos persistentes.
Replication lag	Lecturas antiguas en réplicas	Lecturas críticas desde primary y monitoreo de lag.

6.5 Reflexión y lecciones aprendidas

Una arquitectura híbrida no debe verse como una suma de motores, sino como una distribución intencional de responsabilidades. El aprendizaje principal es que la decisión de persistencia debe basarse en el comportamiento del dato, el riesgo del negocio y la evidencia de rendimiento. Los módulos críticos requieren CP, mientras que

los módulos de experiencia y analítica pueden operar bajo AP con consistencia eventual.

También se evidenció que el rendimiento no depende solo del motor, sino de índices, particiones, selectividad, red, región, límites del proveedor y estrategia de conexión. En especial, el punto de quiebre con 25 usuarios concurrentes mostró la necesidad de incorporar pooling y monitoreo desde etapas tempranas.

7. Recomendaciones estratégicas

7.2 Plan de escalamiento 10x

Tabla 11. Plan de escalamiento para crecimiento 10x.

Capa	Acción recomendada
PostgreSQL	PgBouncer, réplicas de lectura, particionamiento, tuning y plan productivo.
MongoDB	Sharding, índices compuestos y tier M10/M20 o superior.
Cache	Redis para catálogo, sesiones y resultados frecuentes.
Search	OpenSearch o Elasticsearch para búsqueda avanzada.
Sync	Kafka, Debezium o Change Data Capture.
Observabilidad	Prometheus, Grafana, Atlas Monitoring y alertas.
Analítica	Data warehouse o réplicas para reporting pesado.

7.3 Migración de free tier a producción

La migración a producción requiere aumentar capacidad, controlar conexiones y separar cargas. En Supabase o PostgreSQL administrado se recomienda un plan con mayor pool de conexiones, backups, PITR, réplicas de lectura y configuración de PgBouncer. En MongoDB Atlas se recomienda migrar a un tier dedicado, seleccionar región cercana a los usuarios, habilitar monitoreo y configurar alertas. Las credenciales deben moverse a variables de entorno o gestores de secretos.

7.4 Tecnologías complementarias

Tabla 12. Tecnologías complementarias recomendadas.

Tecnología	Uso recomendado
Redis	Cache de catálogo, sesiones y resultados frecuentes.
OpenSearch/Elasticsearch	Búsqueda textual avanzada y facetas.
Kafka o Debezium	Eventos, CDC y sincronización confiable.
Prometheus/Grafana	Observabilidad, métricas y alertas.
Sentry o similar	Trazabilidad de errores de aplicación.
dbt o warehouse	Transformaciones analíticas y modelos de reporting.

7.5 Estrategia CI/CD para cambios de esquema

La estrategia de CI/CD debe incluir migraciones SQL versionadas, validación automática de DDL, pruebas de rollback, validación de JSON Schema MongoDB, ejecución de pruebas de integración y revisión de planes EXPLAIN para consultas críticas. Además, se recomienda separar ambientes de desarrollo, staging y producción, y exigir aprobación técnica para cambios en tablas críticas como orders, payments e inventories.

8. Conclusiones

El proyecto cumplió el objetivo de diseñar, implementar y evaluar una arquitectura híbrida de datos para Ecommify. PostgreSQL permitió gestionar con consistencia fuerte los módulos críticos de negocio, mientras MongoDB aportó flexibilidad para catálogo, comportamiento, recomendaciones, reseñas y búsquedas.

La evidencia empírica permitió demostrar mejoras significativas en PostgreSQL mediante índices especializados y particionamiento. Las pruebas de concurrencia

mostraron mejor desempeño de PostgreSQL en escenarios controlados, mientras que MongoDB mantuvo ventajas arquitectónicas en dominios flexibles y AP. La identificación del punto de quiebre con 25 usuarios concurrentes permitió formular recomendaciones realistas de producción.

La arquitectura 100 % relacional sería más rígida para datos flexibles y eventos. La arquitectura 100 % NoSQL elevaría el riesgo en pagos, órdenes e inventario. Por tanto, la arquitectura híbrida PostgreSQL + MongoDB es la decisión más equilibrada para Ecommify, siempre que se complemente con sincronización robusta, observabilidad, caching, búsqueda especializada y una estrategia de escalamiento gradual.

9. Referencias

- Amazon Web Services. (s. f.). AWS well-architected framework.
<https://aws.amazon.com/architecture/well-architected/>
- Brewer, E. A. (2000). Towards robust distributed systems. Proceedings of the ACM Symposium on Principles of Distributed Computing.
- MongoDB. (s. f.). MongoDB manual. <https://www.mongodb.com/docs/manual/>
- MongoDB. (s. f.). MongoDB Atlas documentation.
<https://www.mongodb.com/docs/atlas/>
- Olist. (s. f.). Brazilian e-commerce public dataset by Olist. Kaggle.
<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation.
<https://www.postgresql.org/docs/>
- Supabase. (s. f.). Supabase documentation. <https://supabase.com/docs>

10. Anexos

- Anexo A. README_POSTGRESQL.md.
- Anexo B. README_MONGODB.md.
- Anexo C. README_SYNC.md.
- Anexo D. README_EVALUACION_CRITICA.md.
- Anexo E. postgresql_performance_results.csv.
- Anexo F. performance_concurrency_results.csv.
- Anexo G. scalability_dataset_growth_results.csv.
- Anexo H. collection_counts.csv.
- Anexo I. Capturas de evidencia de ejecución, índices, EXPLAIN, gráficos y validaciones.